

PRÁTICA 1: Assembly (instruções aritméticas, transferência de dados e desvio)

Atividade individual

I. INSTRUÇÕES

- Na aula de apresentação, os alunos deverão explicar as principais decisões tomadas durante a codificação e executar os programas de modo a mostrar o funcionamento correto de cada um e/ou possíveis fontes de erros.
- Arquivo compactado nomeado "praticaX_nome_aluno.zip", onde X representa o número da prática entregue.
- Esse arquivo deve conter uma pasta com todos os arquivos de código-fonte dos programas em assembly gerados (*.asm) e o relatório (*.pdf). Após o upload do arquivo, faça o download e certifique-se de que o arquivo descompacta e que os arquivos são abertos corretamente.
- Também deverá ser entregue um arquivo *.pdf com imagens de tela da execução dos programas assembly mostrando o funcionamento correto de cada um. Escreva também um ou dois parágrafos explicando as principais decisões tomadas e o que deve ser evidenciado nas imagens de tela.

II. ATENÇÃO

- Todos os programas devem ser desenvolvidos usando **apenas** o conjunto de instruções do MIPS apresentado a seguir. Sim, é possível 😊

MIPS assembly language				
Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1, \$s2, \$s3	\$s1 = \$s2 + \$s3	Three operands; data in registers
	subtract	sub \$s1, \$s2, \$s3	\$s1 = \$s2 - \$s3	Three operands; data in registers
	add immediate	addi \$s1, \$s2, 100	\$s1 = \$s2 + 100	Used to add constants
Data transfer	load word	lw \$s1, 100(\$s2)	\$s1 = Memory[\$s2 + 100]	Word from memory to register
	store word	sw \$s1, 100(\$s2)	Memory[\$s2 + 100] = \$s1	Word from register to memory
Conditional branch	branch on equal	beq \$s1, \$s2, 25	if (\$s1 == \$s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1, \$s2, 25	if (\$s1 != \$s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1, \$s2, \$s3	if (\$s2 < \$s3) \$s1 = 1; else \$s1 = 0	Compare less than; for beq, bne
	set less than immediate	slti \$s1, \$s2, 100	if (\$s2 < 100) \$s1 = 1; else \$s1 = 0	Compare less than constant
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	\$ra = PC + 4; go to 10000	For procedure call

III. UM POUCO DE TEORIA

- Declaração de vetor e armazenamento de dados em memória: após a diretiva **.data**, use o tipo de variável **“word”** para declarar um vetor. Por exemplo:
 - listA: .word 0:9, especifica um vetor de 10 palavras nomeado como listA (*label*).
- Inicialização da memória: Antes de executar o código, podem ser feitas manualmente no campo *“Data Segment”* ou durante a execução do código, via instruções de load store.

```
.data
vetor: .word 0:4      # declaração de vetor de 5 posições
.text
la    $t0, vetor      # lê o valor de endereço do início do vetor
addi  $t2, $zero, 1    # move o valor imediato para $t2
sw    $t2, 0($t0)      # armazena o valor de $t2 na 1a posição do vetor
addi  $t2, $zero, 2    # move o valor imediato para $t2
sw    $t2, 4($t0)      # armazena o valor de $t2 na 2a posição do vetor
addi  $t2, $zero, 3    # move o valor imediato para $t2
sw    $t2, 8($t0)      # armazena o valor de $t2 na 3a posição do vetor
addi  $t2, $zero, 4    # move o valor imediato para $t2
sw    $t2, 12($t0)     # armazena o valor de $t2 na 4a posição do vetor
addi  $t2, $zero, 5    # move o valor imediato para $t2
sw    $t2, 16($t0)     # armazena o valor de $t2 na 5a posição do vetor
```

- As instruções BEQ (*Branch if Equal*) e BNE (*Branch if Not Equal*) são comandos de desvio condicional em assembly MIPS. O BEQ desvia o fluxo do programa se dois registradores forem iguais, enquanto o BNE desvia se forem diferentes. Ambos são essenciais para criar estruturas como *if*, *else* e laços de repetição (*loops*).

IV. O QUE DEVE SER FEITO E APRESENTADO NA AULA DO DIA 21/08/2026 – NÃO É NECESSÁRIO ENTREGAR CÓDIGO E RELATÓRIO PARA ESSA PARTE

1. Faça um programa que receba dois números informados pelo usuário e **retorne 1** como saída 1, se a subtração do primeiro número digitado pelo segundo número for igual a 0, caso contrário, **retorne 0** como saída.
2. Faça um programa que receba 3 números, armazene-os (obrigatoriamente) na ordem de leitura em um vetor V nas posições 0, 1 e 2, troque o valor de V[0] com V[2] em memória, e depois, imprima o novo valor de V[2] na tela (não é para usar estruturas de loop).

V. O QUE DEVE SER FEITO EM CASA E ENTREGAR CÓDIGO JUNTAMENTE COM RELATÓRIO NO MOODLE ATÉ A PRÓXIMA QUINTA-FEIRA 27/08/2026 ÀS 23H59

1. Faça um programa que receba do usuário um número e retorne o somatório de todos os números positivos menores ou iguais a ele.

2. Faça um programa que receba do usuário um número, para representar o número de termos (N) de uma sequência. A saída deve mostrar a seguinte sequência de valores na ordem crescente 1 2 4 6 8 10 12..., limitada por N termos.
3. Suponha o trecho de código a seguir escrito em linguagem de alto nível. Transcreva-o para linguagem assembly MIPS.

```
int v[20];
for (int i=0; i< 20; i++){
    cout << "Informe um número ";
    cin >> v[i];
}

int i = 0;
int x = 0;
int n = 20;
while (i < n) {
    x = x + v[n-1];
    v[i] = x;
    i++;
}
cout << "Novos valores de V: ";
for (int i=0; i< 20; i++){
    cout << v[i] << " ";
}
```

4. Implemente o algoritmo da divisão usando somente operações de soma e subtração, conforme mostrado a seguir.

Algoritmo da divisão usando somente operações de soma e subtração

```
dividendo = int(input("Dividendo: "))
divisor = int(input("Divisor: "))
quociente = 0
x = dividendo
while x >= divisor:
    x = x - divisor
    quociente = quociente + 1
resto = x
print(f"{dividendo} / {divisor} = {quociente} (quociente) {resto} (resto)")
```